

Tszh-Agent全流程开发思路

腾昇智和 · 一键 Video Workspace — 全流程开发记录

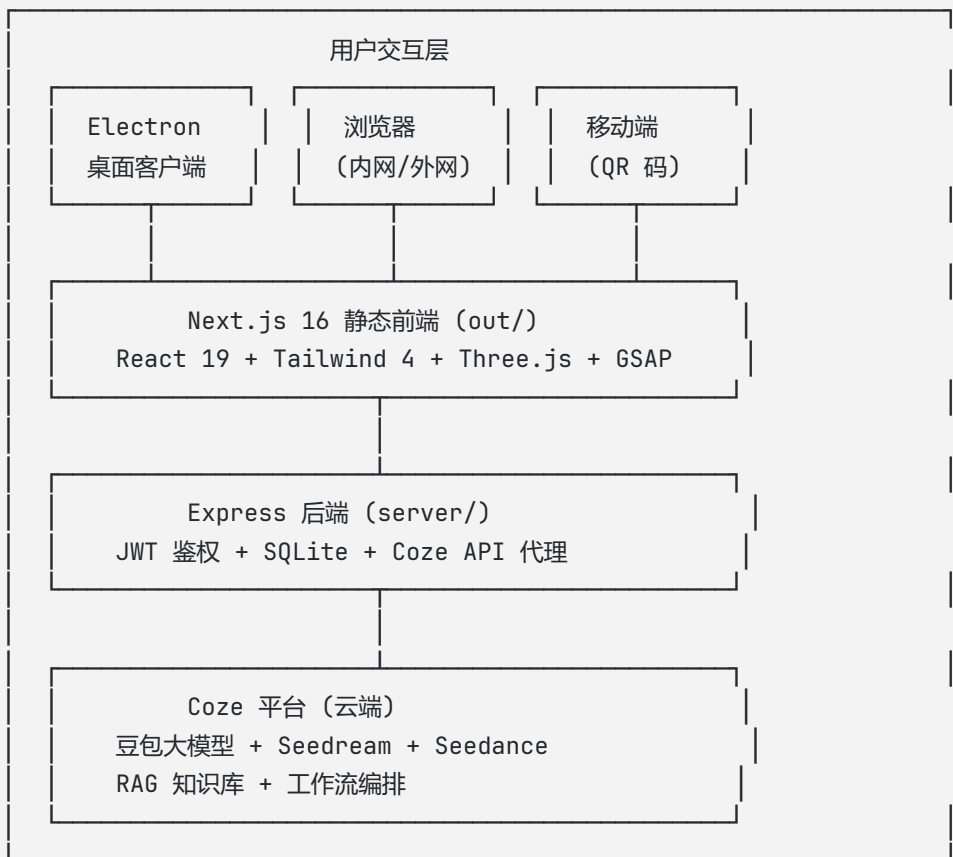
项目定位：基于 Coze 平台的 AI 短视频全链路自动生成智能体前端交互系统
开发周期：2026 年 4 月底 — 2026 年 6 月上
技术栈：Next.js 16 + React 19 + Tailwind CSS 4 + Three.js + Express + SQLite + Electron
AI 平台：Coze（豆包大模型 + Seedream 生图 + Seedance 生视频）
最后更新：2026-06-08

目录

- 1. [项目总览与技术选型](#)
- 2. [Coze 智能体核心功能开发](#)
- 3. [智能体人设与逻辑设计](#)
- 4. [工作流功能设计](#)
- 5. [RAG 知识库设计](#)
- 6. [前端功能开发](#)
- 7. [后端功能开发](#)
- 8. [数据库设计](#)
- 9. [UI 设计系统](#)
- 10. [视觉设计与动效](#)
- 11. [内网前后端服务全流程](#)
- 12. [Electron 打包流程](#)
- 13. [ECS 云服务器 + 外网部署](#)
- 14. [开发过程中遇到的问题与解决方案](#)

1. 项目总览与技术选型

1.1 项目架构



1.2 技术选型对比

前端框架

方案	优点	缺点	选择
Next.js 16 (App Router)	React 19 支持、静态导出、Turbopack 编译快	SSR 对纯客户端应用多余	✅ 选用
Vite + React	极快的 HMR、轻量	缺少元框架能力、路由需手动配	❌
Nuxt 3	Vue 生态、全栈能力强	团队更熟悉 React	❌

最终选择 Next.js 16，使用 `output: "export"` 纯静态导出模式，放弃 SSR，所有页面为客户端组件。

React 19 相当于给你装上了能自动优化代码的“智能大脑”，大幅简化异步操作；静态导出好比在发布时一次性把所有页面打印成 HTML “快照”，让网站首屏飞快、托管省钱且利于 SEO；而 Turbopack 则是给开发服务器换上了“超级引擎”，实现按需编译和增量更新，让大型项目的冷启动从十几秒缩短到一两秒。三者合力，让开发者写得更少、跑得更快、部署更轻松。

CSS 方案

方案	优点	缺点	选择
Tailwind CSS 4	原子化、@theme 内联、CSS 变量原生支持	学习曲线	✅ 选用
CSS Modules	原生、无依赖	主题切换复杂	❌
Styled Components	JS 中写 CSS、动态样式	运行时开销、React 19 兼容问题	❌

3D/粒子效果

方案	优点	缺点	选择
Three.js + @react-three/fiber	React 声明式 3D、生态丰富	包体积大	✅ 选用
Canvas 2D	轻量、简单	3D 效果受限	部分使用 (SplashScreen)
WebGL 原生	性能最优	开发成本极高	❌
PixiJS	2D 渲染强	3D 支持弱	❌

动画库

方案	用途	选择
GSAP	复杂时间线编排 (SplashScreen 爆炸、ScrollTrigger)	✅ 主力
anime.js	微交互 (按钮 hover、数值动画)	✅ 辅助
Framer Motion	React 声明式动画	❌ 包体积大
CSS Animations	简单循环动画	✅ 基础层

后端

方案	优点	缺点	选择
Express 5	成熟、中间件丰富、轻量	无 TypeScript 原生支持	✅ 选用
Fastify	性能更好、TypeScript 支持	生态不如 Express	❌
Next.js API Routes	与前端一体	静态导出模式下不可用	❌
NestJS	企业级、模块化	过重、学习成本高	❌

数据库

方案	优点	缺点	选择
SQLite (better-sqlite3)	零配置、文件级、同步 API、性能好	不支持并发写入 (WAL 缓解)	✅ 选用
PostgreSQL	功能强大、并发好	需要额外服务、部署复杂	❌
IndexedDB	浏览器原生	查询复杂、无服务端访问	❌
localStorage	简单	容量限制 (5MB)、无结构化查询	作为缓存层

桌面打包

方案	优点	缺点	选择
Electron	成熟、跨平台、完整 Node.js 环境	包体积大 (~100MB)	✅ 选用
Tauri	极小体积、Rust 后端	需要 Rust、Windows 打包需 WebView2	❌
Neutralinojs	轻量	生态弱、社区小	❌

2. Coze 智能体核心功能开发

Coze (扣子) 是整个项目的 AI 大脑。用户在前端输入自然语言创意，Coze 平台上的智能体负责理解意图、检索知识库、编排分镜、调用生图/生视频模型，最终返回结构化的视频/图片结果。这一章记录了 Coze 平台的选型过程、API 对接细节、以及如何处理 Coze 特有的响应格式。

2.1 平台选择

选择 Coze (扣子) 平台的原因：

对比维度	Coze	自建 LLM 服务	Dify	FastGPT
workflow编排	✅ 可视化拖拽	需自研	✅ 可视化	✅ 可视化
知识库	✅ 表格+文本	需自建向量库	✅	✅
生图/生视频	✅ 原生集成 Seedream/Seedance	需对接第三方	❌ 需插件	❌
部署成本	免费额度充足	GPU 服务器昂贵	需自部署	需自部署
团队协作	✅ 多人编辑	N/A	✅	✅

核心优势：Coze 原生集成了字节跳动的 Seedream (图像生成) 和 Seedance (视频生成) 模型，无需额外对接。

2.2 Agent 配置

- **决策模型：**Doubao 2.0 pro
- **图像生成：**Seedream 4.5
- **视频生成：**Seedance 1.0 pro
- **API 端点：**POST `https://api.coze.cn/v3/chat` (SSE 流式)

2.3 Coze API 调用链路

用户输入 → 前端 ChatFlow → POST `/api/agent/stream`
→ Express 后端代理 → Coze `/v3/chat` (SSE)

→ 豆包大模型决策 → 调用 Seedream/Seedance workflow
→ SSE 流式返回 → 前端逐字渲染

2.4 Coze 卡片响应解析

Coze 的知识库检索结果以卡片格式返回，需要特殊解析：

```
function parseCozeCard(text) {  
  const json = JSON.parse(text);  
  if (json.card_type && json.data) {  
    const cardData = typeof json.data === "string" ? JSON.parse(json.data) :  
    json.data;  
    // 从 variables 中提取知识库结果  
    if (cardData.variables) {  
      for (const [key, variable] of Object.entries(cardData.variables)) {  
        if (variable.defaultValue && Array.isArray(variable.defaultValue)) {  
          // 解析 con 字段中的 JSON  
          for (const item of variable.defaultValue) {  
            const conData = JSON.parse(item.con);  
            // 提取中文参数值、英文 Prompt 值、分类、效果说明等  
          }  
        }  
      }  
    }  
  }  
}
```

2.5 SSE 流式响应处理

```
// 三种事件类型  
const COZE_EVENTS = {  
  MESSAGE_DELTA: "conversation.message.delta", // 逐字流式  
  MESSAGE_COMPLETED: "conversation.message.completed", // 单条完成  
  CHAT_COMPLETED: "conversation.chat.completed", // 整体完成  
};  
  
// 前端处理逻辑  
source.addEventListener("delta", (e) => {  
  const { content } = JSON.parse(e.data);  
  // 追加到当前消息文本  
});  
  
source.addEventListener("done", (e) => {  
  const { videoUrl, imageUrls, text } = JSON.parse(e.data);
```

```
// 渲染最终结果卡片
});

source.addEventListener("follow_up", (e) => {
  const { suggestions } = JSON.parse(e.data);
  // 显示后续建议气泡
});
```

3. 智能体人设与逻辑设计

智能体的人设不是一个简单的 system prompt，而是一套完整的决策逻辑体系。它定义了智能体如何理解用户意图、如何分配镜头、如何确保多镜头一致性、以及何时停下来追问。这套设计直接影响了最终生成视频的质量和用户体验。

3.1 角色定位

智能体角色为「国际顶级视觉导演 × AI 短视频导演 × 分镜编剧 × 图像/视频 Prompt 编排专家 × 制作模式路由决策引擎」的融合体。

核心哲学三支柱：

1. **纯粹美学至上** — 去除机械装甲、动物耳朵等杂乱元素
2. **叙事为王** — 每个镜头必须服务于情感或故事张力
3. **控制变量** — 可复用的角色锚定描述，锁定多镜头一致性

3.2 四维视角输出框架

每次响应必须流经四个视角：

视角	职责	输出内容
制片人	意图重构	情感核心、视觉调性、硬约束
编剧	叙事骨架	三幕式/起承转合剧本，需用户确认
导演	镜头策略	镜头设计、运镜策略、一致性控制点
工程师	执行路径	选择三种工作流模式之一

3.3 八维覆盖扫描

系统内部对 8 个维度进行感知扫描：

维度	说明
主体 (Subject)	人物/物体身份
场景 (Scene)	环境、背景
风格 (Style)	视觉风格、艺术流派
光影 (Lighting)	光源、明暗对比
色彩 (Color)	色调、饱和度
情感 (Emotion)	情绪氛围
镜头 (Camera)	景别、运镜
叙事 (Narrative)	故事节奏、张力

- 6+ 维度明确 → 直接生成
- 3-5 维度 → 从知识库自动补全
- <3 维度 → 温和追问 1-2 个问题

3.4 渐进式输入拦截

如果用户提供了故事但未描述主角外貌，系统**必须停下来询问**后才生成分镜。这是「控制变量」原则的硬性执行。

3.5 口语化→术语查询重写

用户输入往往是口语化的（如“镜头慢慢推进”），系统需要：

1. 路由到 KB_Bridge 文本知识库
2. 提取 [MAP] 条目中的标准化术语
3. 用术语进行第二阶段表格知识库检索

4. workflow 功能设计

workflow 是 Coze 平台上的可视化编排引擎，决定了从“分镜脚本”到“视频产物”的执行路径。项目设计了三种 workflow 模式，分别对应不同的创作风格。其中“拉链拼接”架构是为了解决连贯模式的串行超时问题而发明的，是整个 workflow 设计中最具技术含量的部分。

4.1 三种 workflow 模式

模式	适用场景	特点
经典混剪模式	跳剪编辑、逻辑连续	无需特定主角
IP 锁定混剪模式	有特定主角的项目	包含角色妆造子 workflow
连贯模式	无缝镜头过渡、“一镜到底”感	统一时空光影

4.2 工作流 A：混剪模式

核心架构：四模式计算调度网关

LLM 对每个镜头输出布尔标记：

- `s_img` = 需要起始帧图像
- `e_img` = 需要结束帧图像
- `reference_img` = 需要参考图像
- `is_camera_fixed` = 固定机位

IF 选择器动态激活所需的 Seedream 生图节点和 Seedance 生视频节点，避免浪费 API 调用。

四种制作模式：

模式	布尔组合	用途
仅起始帧	<code>s_img=true</code>	快节奏、新镜头入场
起始+结束帧	<code>s_img+e_img</code>	状态转换（日转夜、远转近）
仅参考图	<code>reference_img=true</code>	氛围静帧、情感特写
仅结束帧	<code>e_img=true</code>	海报定格、高潮揭示

输出 Schema（每镜头 9 字段）：

```
{
  "scene_id": 1,
  "visual_prompt_cn": "中文视觉描述",
  "camera_motion_cn": "秒级精确时间线运镜脚本",
  "video_ratio": "16:9",
  "video_duration": 5,
  "is_camera_fixed": false,
  "reference_img": false,
  "s_img": true,
  "e_img": false
}
```

4.3 工作流 B：连贯模式

核心架构："拉链拼接" (Zipper Stitching)

传统串行方式会超时，纯并行会死锁。解决方案三阶段解耦：

1. **数据立交桥：**将图像描述和运镜指令分离
2. **核心"变速箱"代码节点：**用滑动窗口将 N 张图配对为 N-1 个视频任务
3. **视频批量全并行渲染：**零交叉依赖

```
# 拉链拼接核心逻辑
scenes = input_data["scenes"]
image_descs = [s["visual_prompt_cn"] for s in scenes]
camera_motions = [s["camera_motion_cn"] for s in scenes]

video_tasks = []
for i in range(len(scenes) - 1):
    video_tasks.append({
        "start_frame_desc": image_descs[i],
        "end_frame_desc": image_descs[i + 1],
        "camera_motion": camera_motions[i],
        "video_duration": scenes[i]["video_duration"],
    })
```

简化输出 Schema (每镜头 6 字段)：

```
{
  "scene_id": 1,
  "visual_prompt_cn": "静态视觉锚点",
  "camera_motion_cn": "运镜指令",
  "video_ratio": "16:9",
  "video_duration": 5,
  "is_camera_fixed": false
}
```

4.4 StylePromptMaster 插件

在生成英文 Prompt 时，分三档增强：

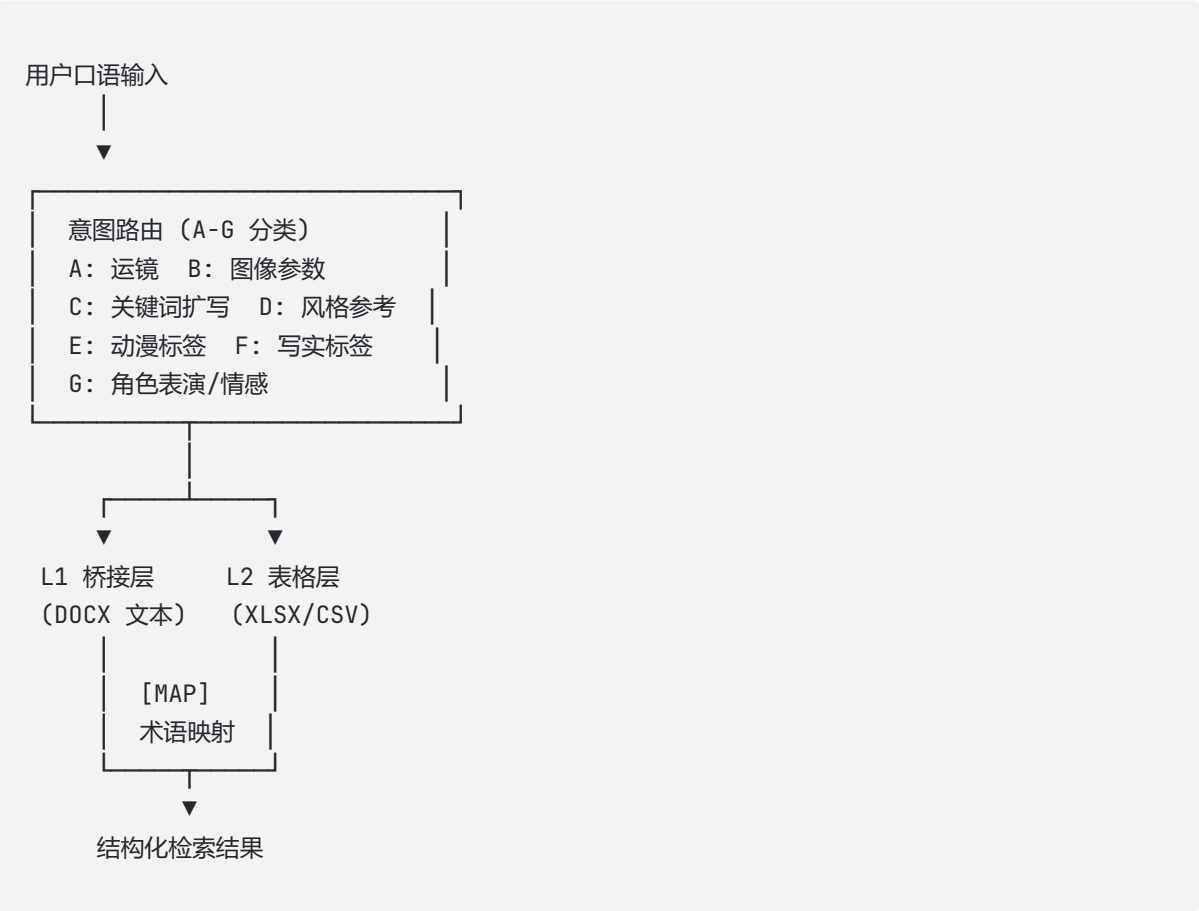
- **Standard：**标准增强
- **Restrained：**克制风格
- **Dramatic：**戏剧化风格

同时保持角色锚定描述的一致性。

5. RAG 知识库设计

知识库是智能体的“专业记忆”——它让 AI 不再凭空想象参数，而是基于经过验证的专业数据生成 Prompt。项目构建了“四库一桥”检索架构，其中最大的教训是：Coze 的表格知识库不是纯向量检索，而是 NL2SQL + 多路召回融合，这个认知差导致了前三版知识库的命中率都低于 50%。

5.1 “四库一桥”检索架构



5.2 四个知识库

知识库	格式	条目数	字段数	分类数
图像 Prompt 参数表	XLSX	335	14	12
关键词扩写细节库	XLSX	100	13	9
运镜知识库	CSV	30 (16 基础 + 14 组合)	12	-
Midlibrary 艺术家与风格	XLSX	15 (精选)	15	-

5.3 桥接层 (KB_Bridge)

三个 DOCX 文档，将口语化查询映射到 [MAP] 条目：

```
[MAP] slot_key: camera_push_in
用户可能说: "镜头慢慢推进" / "推进" / "靠近" / "zoom in"
英文值: push in, zoom in
适用场景: 强调主体、制造压迫感、聚焦细节
```

检索流程：

1. 用户说"镜头慢慢推进"
2. 路由到运镜桥接文档
3. 匹配 [MAP] 条目，提取 slot_key: camera_push_in
4. 用 camera_push_in 查表格运镜知识库
5. 返回完整的运镜参数（英文 Prompt、时间线脚本、适用场景）

5.4 Coze 表格知识库的 NL2SQL 机制

关键教训：Coze 表格知识库不是纯向量检索，而是 **NL2SQL + 多路召回融合**。

平台内部流程：

```
导入 → 解析 → 索引 → 用户查询 → 检索（字段匹配 + 语义相似） → 上下文组装 → 豆包生成
```

影响命中率的 8 个关键因素：

1. 列名清晰（中英文双语）
2. 每行一个知识单元
3. 每列一个语义维度
4. 双语别名
5. 无重复行
6. 查询可映射到表条件
7. 系统提示词约束知识库使用
8. 召回数量与上下文长度平衡

5.5 命中率优化工具

开发了 17 个 Python 脚本进行知识库优化：

脚本	功能
测试KB命中率.py / _v2.py	测试知识库命中率
测试docx分片命中率.py	测试 DOCX 分片检索效果
优化docx分片.py	优化 DOCX 文件分片策略
优化KB_Bridge分片.py	优化桥接文档分片
重构并测试KB.py	重构知识库结构并验证
补词并终测.py	补充缺失术语并最终测试
扩充Midlibrary艺术家库.py	扩充艺术家知识库

5.6 本地 RAG 服务

独立的 Python RAG 服务（rag-service/）：

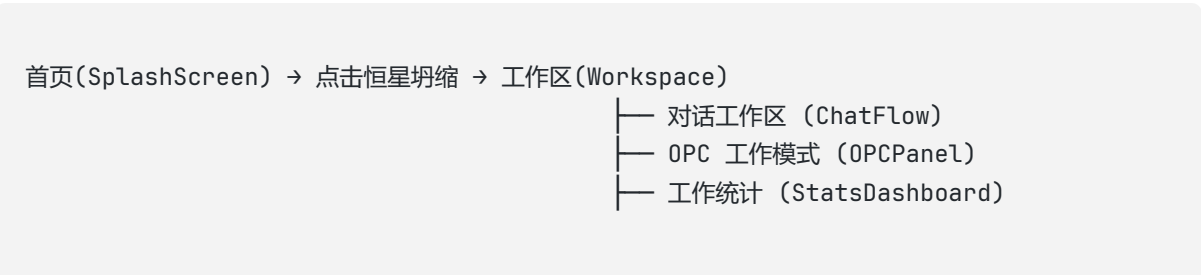
```
# rag-client.ts 调用方式
const response = await fetch("http://localhost:5000/rag/retrieve", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ query, topK: 5, scoreThreshold: 0.45 }),
  signal: AbortSignal.timeout(3000), // 3 秒超时
});
```

意图检测过滤：问候语、摘要请求、“继续”等不需要检索的消息会跳过 RAG 调用。

6. 前端功能开发

前端是用户与 AI 交互的唯一界面。它需要同时处理实时流式对话、3D 粒子背景动画、多页面路由、离线优先存储、以及与后端的双向同步。整个前端采用“全客户端组件”架构（无 SSR），通过 localStorage 实现离线可用，通过 fire-and-forget 模式实现在线同步。

6.1 页面架构



└─ LibTV (AI 生图/生视频)
└─ 画廊 (Gallery)

五页路由采用客户端 state 切换 (非 URL 路由), 配合 `PageTransition` 组件实现 CSS 过渡动画。

6.2 ChatFlow 对话组件

双 API 策略:

- 纯文本消息 → SSE 流式 (`/api/agent/stream`), 逐字渲染
- 含文件上传 → 非流式 (`/api/agent`), 等待完整响应

Session 管理:

- localStorage 键: `tszh_sessions` (会话列表)、`tszh_msgs_{id}` (消息)
- 300ms 防抖保存
- 登录后自动同步服务器

思维阶段动画:

```
const stages = [  
  "正在理解你的创意意图...",  
  "检索知识库, 匹配最佳参数...",  
  "构建分镜脚本...",  
  "生成视觉画面...",  
];  
// 每 2 秒切换一次
```

结果归一化:

```
function normalizePayload(data) {  
  // 统一处理各种响应格式为 { payload, text } 形状  
  // 支持 videoUrl、imageUrls、raw.text 等多种返回格式  
}
```

6.3 OPC 创作助手面板

快速模板: 6 大类 (产品短片、品牌故事、社媒内容、创意实验、教学、活动推广)

风格预设: 70+ 风格, 8 大分类 (东方美学、电影导演、摄影、科幻、艺术运动、动画、商业、当代)

运镜选择：基础运镜 + 组合运镜，点击插入到 Prompt 编排区

参数芯片：分类参数芯片，点击插入到光标位置

模板管理：用户自定义模板的 CRUD，存储在 localStorage

6.4 混合持久化策略

写入流程：

localStorage（立即）→ 异步写入服务器（fire-and-forget）

读取流程：

登录时 → 从服务器拉取 → 与本地合并 → 写回 localStorage

双写模式的目的是：离线可用 + 在线同步。

6.5 SSE 流式解析

```
const source = new EventSource("/api/agent/stream");

source.addEventListener("delta", (e) => {
  const { content } = JSON.parse(e.data);
  setMessages((prev) => {
    const last = prev[prev.length - 1];
    return [...prev.slice(0, -1), { ...last, text: last.text + content }];
  });
});

source.addEventListener("done", (e) => {
  const data = JSON.parse(e.data);
  // 渲染视频/图片结果卡片
  source.close();
});
```

7. 后端功能开发

后端是连接前端与 Coze AI 平台的桥梁。它负责三件事：(1) 代理 Coze API 调用并解析 SSE 流式响应；(2) 提供用户鉴权、对话存储、模板管理等业务 API；(3) 在 Electron 模式下同时承担静态文件服务器的角色。为了适配两种运行环境（独立部署 + Electron 内嵌），设计了双服务器架构。

7.1 双服务器架构

服务器	文件	用途	路由来源
独立服务器	server/server.js	开发/云部署	外部路由模块
Electron 服务器	server/server-express.js	桌面客户端	内联路由 + 外部路由模块

7.2 Coze API 代理

```
// SSE 流式代理
app.post("/api/agent/stream", async (req, res) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");

  const resp = await fetch(`${baseUrl}/v3/chat`, {
    method: "POST",
    headers: { Authorization: `Bearer ${token}`, "Content-Type":
"application/json" },
    body: JSON.stringify({
      bot_id: botId,
      user_id: userId,
      stream: true,
      auto_save_history: true,
      additional_messages: buildMessages(prompt, history),
    }),
    signal: AbortSignal.timeout(10 * 60 * 1000), // 10 分钟超时
  });

  // 解析 Coze SSE 并转发给客户端
  // delta → 逐字流
  // completed → 最终结果
  // chat.completed → 完成信号
});
```

7.3 JWT 鉴权

```
// 非阻塞 JWT 提取中间件
app.use((req, res, next) => {
  req.user = null;
  const authHeader = req.headers.authorization;
  if (authHeader && authHeader.startsWith("Bearer ")) {
    try {
      const token = authHeader.slice(7);
```

```

    req.user = jwt.verify(token, JWT_SECRET);
  } catch {}
}
next();
});

// 仅保护 /api/agent 路由 (调 Coze 花钱)
app.use("/api/agent", (req, res, next) => {
  if (req.user) return next();
  if (!API_SECRET_KEY) return next();
  const key = req.headers["x-api-key"];
  if (key !== API_SECRET_KEY) return res.status(401).json({ error: "Invalid API key" });
  next();
});

```

7.4 媒体 URL 提取

```

const MEDIA_URL_RE = /https?:\/\/[^\s"'<>]+\.(mp4|mov|avi|webm|jpg|jpeg|png|webp|gif)/gi;

function extractMediaUrls(text) {
  // 先尝试 JSON 解析
  if (text.startsWith("{") || text.startsWith "[")) {
    const json = JSON.parse(text);
    if (json.videoUrl || json.video_url) return { videoUrl: ... };
  }
  // 回退到正则匹配
  const matches = text.match(MEDIA_URL_RE);
  // 分类为视频和图片
}

```

7.5 并发控制

```

let activeRequests = 0;
const MAX_CONCURRENT = 5;

app.post("/api/agent", async (req, res) => {
  if (activeRequests >= MAX_CONCURRENT) {
    return res.status(429).json({ error: "服务器繁忙，请稍后再试" });
  }
  activeRequests++;
  try {
    // 处理请求

```

```

    } finally {
      activeRequests--;
    }
  });

```

8. 数据库设计

数据库选择了 SQLite (better-sqlite3)，这是一个“有争议但适合本项目”的决定。争议在于 SQLite 不适合高并发写入；适合在于它是零配置的文件级数据库，Electron 打包后不需要额外安装数据库服务，每个用户的数据库就是本地一个 `.db` 文件。通过 WAL 模式和合理的索引设计，在 10 人以下的使用场景中性能完全够用。

8.1 表结构 (12 张表)

```

-- 用户表
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL,          -- bcrypt 哈希
  display_name TEXT DEFAULT '',
  created_at TEXT DEFAULT (datetime('now')),
  updated_at TEXT DEFAULT (datetime('now'))
);

-- 对话表
CREATE TABLE conversations (
  id TEXT PRIMARY KEY,              -- UUID
  user_id INTEGER NOT NULL,
  title TEXT DEFAULT '新对话',
  created_at TEXT, updated_at TEXT,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- 消息表
CREATE TABLE messages (
  id TEXT PRIMARY KEY,
  conversation_id TEXT NOT NULL,
  role TEXT NOT NULL CHECK(role IN ('user', 'agent')),
  content TEXT DEFAULT '',
  payload TEXT,                     -- JSON 格式的结构化数据
  is_error INTEGER DEFAULT 0,
  error_text TEXT DEFAULT '',
  created_at TEXT,

```

```

FOREIGN KEY (conversation_id) REFERENCES conversations(id) ON DELETE CASCADE
);

-- 模板表
CREATE TABLE templates (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  category TEXT NOT NULL,
  icon TEXT DEFAULT '+',
  label TEXT NOT NULL,
  prompt TEXT NOT NULL,
  created_at TEXT,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- 生成记录表
CREATE TABLE generations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  conversation_id TEXT,
  prompt TEXT NOT NULL,
  video_url TEXT,
  image_urls TEXT,                -- JSON 数组
  raw_text TEXT,
  status TEXT DEFAULT 'completed',
  coze_conversation_id TEXT,
  tokens_used INTEGER DEFAULT 0,
  created_at TEXT,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- 用户设置表
CREATE TABLE user_settings (
  user_id INTEGER PRIMARY KEY,
  theme TEXT DEFAULT 'dark',
  language TEXT DEFAULT 'zh',
  opc_style TEXT DEFAULT '',
  opc_params TEXT DEFAULT '{}',  -- JSON
  created_at TEXT, updated_at TEXT,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- OPC 会话表
CREATE TABLE opc_sessions (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL DEFAULT '新对话',
  created_at INTEGER NOT NULL DEFAULT (unixepoch()),
  updated_at INTEGER NOT NULL DEFAULT (unixepoch()),

```

```

    message_count INTEGER DEFAULT 0,
    summary TEXT DEFAULT NULL
);

-- OPC 消息表
CREATE TABLE opc_messages (
    id TEXT PRIMARY KEY,
    session_id TEXT NOT NULL,
    role TEXT NOT NULL,
    content TEXT NOT NULL DEFAULT '',
    timestamp INTEGER NOT NULL DEFAULT (unixepoch()),
    metadata TEXT DEFAULT NULL,          -- JSON
    FOREIGN KEY (session_id) REFERENCES opc_sessions(id) ON DELETE CASCADE
);

-- OPC 跨会话记忆
CREATE TABLE opc_memory (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL,
    category TEXT DEFAULT 'general',
    updated_at INTEGER NOT NULL DEFAULT (unixepoch())
);

-- LLM 模型配置
CREATE TABLE llm_providers (
    id TEXT PRIMARY KEY,
    config TEXT NOT NULL,                -- JSON
    is_active INTEGER DEFAULT 0,
    updated_at INTEGER NOT NULL DEFAULT (unixepoch())
);

-- 用户偏好
CREATE TABLE user_prefs (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL,
    updated_at INTEGER NOT NULL DEFAULT (unixepoch())
);

-- 通知表
CREATE TABLE notifications (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    message TEXT NOT NULL,
    type TEXT DEFAULT 'info',
    read INTEGER DEFAULT 0,
    created_at INTEGER NOT NULL DEFAULT (unixepoch())
);

```

```
-- 应用设置
CREATE TABLE app_settings (
  key TEXT PRIMARY KEY,
  value TEXT NOT NULL,
  updated_at INTEGER NOT NULL DEFAULT (unixepoch())
);
```

8.2 索引策略

```
CREATE INDEX idx_conv_user ON conversations(user_id);
CREATE INDEX idx_msg_conv ON messages(conversation_id);
CREATE INDEX idx_gen_user ON generations(user_id);
CREATE INDEX idx_gen_date ON generations(created_at);
CREATE INDEX idx_tpl_user ON templates(user_id);
CREATE INDEX idx_opc_msg_session ON opc_messages(session_id, timestamp);
```

8.3 数据库路径策略

```
let DB_PATH;
if (process.env.STZH_DATA_DIR) {
  // Electron 模式: 使用 userData 目录 (可写)
  DB_PATH = path.join(process.env.STZH_DATA_DIR, "stzh.db");
} else if (__dirname.includes(".asar")) {
  // 兜底: asar 内但没有环境变量
  DB_PATH = path.join(path.dirname(__dirname), "stzh.db");
} else {
  // 开发/云服务器模式: 使用当前目录
  DB_PATH = path.join(__dirname, "stzh.db");
}
```

9. UI 设计系统

视觉设计的核心理念是“深空天文台”——用户坐在一座搭建在深空中的天文台控制室里，对话流是观测窗口，AI 是坐在对面的导演。所有 UI 元素都围绕这个隐喻展开：深空底色、旋转星环、粒子辉光、像素风标题。设计系统禁止使用纯黑/纯白、通用字体、玻璃拟态等“安全但平庸”的选择，追求独特的品牌气质。

9.1 色彩系统

变量	色值	用途
--glow-warm	#e89840	暖琥珀，强调、主要交互
--glow-cool	#6088d8	冷靛，辅助、冷色调
--glow-aurora	#9880d0	极光紫，点缀、过渡
--space-deep	#050a14	深空底色
--space-panel	#0a1228	面板背景
--space-surface	#0f1a35	表面/卡片

9.2 11 套内置主题

主题名	特点
深空观测者 (默认)	深蓝黑底 + 暖琥珀强调
原子实验室	科学感、冷色调
量子花园	紫绿渐变、生物感
星云漂流	柔和粉紫、漂浮感
太阳熔炉	烈焰橙红、高温感
水晶洞穴	冰蓝透明、清冷感
虚空信号	青色扫描线、赛博感
锈蚀密室	暖棕铁锈、复古工业
光子场	纯白高亮、极简感
深海虚空	深蓝绿、水下感

主题切换通过 `document.documentElement.dataset.theme` 属性驱动 CSS 变量。

9.3 字体层级

层级	字体	用途
Display 标题	GeistPixel-Line (像素风)	大标题、品牌标签
Display 中文	ZCOOL QingKe HuangYou (毛笔)	品牌中文标题
Body 正文	Geist Sans	正文 (无 CJK, fallback 到系统字体)
Mono 代码	Geist Mono	代码块、数据展示

9.4 Edge-Glow 系统

CSS `@property --glow-angle` 驱动旋转锥形渐变边缘辉光：

```

@property --glow-angle {
  syntax: "<angle>";
  initial-value: 0deg;
  inherits: false;
}

.edge-glow {
  --_glow: var(--glow-warm);
  border: 1px solid transparent;
  background:
    linear-gradient(var(--space-panel), var(--space-panel)) padding-box,
    conic-gradient(from var(--glow-angle), var(--_glow), transparent 30%, var(--_glow)) border-box;
  animation: glow-rotate 4s linear infinite;
}

@keyframes glow-rotate {
  to { --glow-angle: 360deg; }
}

```

四级强度: `.edge-glow` / `.edge-glow-strong` / `.edge-glow-subtle` / `.edge-glow-pulse` / `.edge-glow-sweep`

9.5 Z 轴层级系统

```

:root {
  --z-base: 1;
  --z-content: 10;
  --z-overlay: 20;
  --z-sidebar: 30;
  --z-modal: 40;
  --z-toast: 50;
  --z-max: 100;
}

```

四层背景 (LayerStack) :

1. `env-bg` : 主题背景色
2. `space-texture` : 空间纹理
3. `space-grid` : 网格线
4. `space-noise` : 分形噪点

9.6 禁止清单

- ❌ 玻璃拟态 (backdrop-blur 装饰性使用)
- ❌ emoji 作为功能性图标
- ❌ 纯黑 #000 或纯白 #fff
- ❌ bounce/elastic 缓动 (除非品牌刻意)
- ❌ Inter/Roboto/Arial 等通用字体

10. 视觉设计与动效

动效不是装饰，而是叙事的一部分。SplashScreen 的恒星坍缩动画是用户见到的第一个画面——它用 3 秒时间讲述了一个“恒星爆炸、品牌诞生”的故事。Three.js 粒子系统不只是背景装饰，它会响应用户操作（鼠标吸引/排斥）和系统状态（思考模式旋涡），让整个界面“活”起来。

10.1 SplashScreen 恒星坍缩

Canvas 2D 渲染的全屏动画：

1. **恒星收缩**：500 颗星向中心坍缩
2. **冲击波环**：扩散的光环
3. **文字爆裂**：「腾昇智和」四字像素化后爆裂散开
4. **粒子爆发**：160 粒子 + 200 像素碎片
5. **星云背景**：6 个漂浮星云团
6. **CRT 扫描线**：水平线条叠加

GSAP timeline 编排整个序列，约 3 秒完成。

10.2 Three.js 粒子系统

三层粒子：

- 150 冷色调远景粒子（大散布、低透明度）
- 100 前景粒子
- 50 暖色调近景粒子

特性：

- 星座连线：25 个随机点按距离连线
- 交互物理：鼠标吸引/排斥力，点击排斥，弹簧回弹
- 思考模式旋涡：粒子向中心汇聚
- 生命周期：淡入/淡出 + 重生
- AdditiveBlending：发光效果
- 主题跟随：读取 CSS 变量并每 0.6s 刷新

10.3 CSS 星环

5 层同心圆，纯 CSS 实现：

- 随机尺寸（160-800px）
- 随机转速（10-42s）
- 3D 透视（rotateX/rotateY）
- 中心恒星：90px 径向渐变 + 多层 box-shadow 辉光
- 呼吸动画：每环独立周期
- 发送脉冲：GSAP scale 1.25x 弹性回弹

10.4 鼠标拖尾

Canvas 鼠标拖尾 + 涟漪效果，使用主题色缓存。

10.5 聊天布局

左对称非平衡设计（VARIANCE 8）：

- 消息气泡带 perspective 变换
- 微重力浮动动画（`space-float-a/b/c`）
- Agent 消息暖色调，用户消息强调色
- 思考指示器：呼吸 box-shadow 的弹跳点

11. 内网前后端服务全流程

11.1 开发环境启动

```
# 终端 1: 启动 Next.js 开发服务器
cd D:\fms688_stzh-Agent
npm run dev

# 终端 2: 启动 Express 后端
cd D:\fms688_stzh-Agent\server
node server.js

# 终端 3 (可选) : 启动 RAG 服务
cd D:\fms688_stzh-Agent
npm run rag:start
```

11.2 环境变量配置

前端 `.env.local` (项目根目录) :

```
AGENT_BACKEND_URL=http://localhost:8080
AGENT_API_KEY=你的api密钥
NEXT_PUBLIC_AGENT_BACKEND_URL=http://localhost:8080
```

后端 `server/.env.local` :

```
COZE_API_TOKEN=你的_coze_token
COZE_BOT_ID=你的_bot_id
COZE_USER_ID=stzh_user
COZE_BASE_URL=https://api.coze.cn
JWT_SECRET=随机长字符串
API_SECRET_KEY=
PORT=8080
```

11.3 API 端点总览

方法	路径	说明	鉴权
POST	<code>/api/agent</code>	AI 对话 (JSON 响应)	JWT/API Key
POST	<code>/api/agent/stream</code>	AI 对话 (SSE 流式)	JWT/API Key
POST	<code>/api/auth/register</code>	注册	无
POST	<code>/api/auth/login</code>	登录	无
GET	<code>/api/auth/me</code>	获取当前用户	JWT

方法	路径	说明	鉴权
GET	/api/conversations	获取会话列表	JWT
POST	/api/conversations	创建会话	JWT
GET	/api/conversations/:id	获取会话消息	JWT
PUT	/api/conversations/:id	更新会话标题	JWT
DELETE	/api/conversations/:id	删除会话	JWT
POST	/api/conversations/:id/messages	保存消息	JWT
GET	/api/templates	获取模板列表	JWT
POST	/api/templates	创建模板	JWT
DELETE	/api/templates/:id	删除模板	JWT
GET	/api/generations	获取生成记录	JWT
GET	/api/generations/stats	获取统计数据	JWT
GET/PUT	/api/settings	用户设置	JWT
GET/POST	/api/app-settings	应用设置	无
GET/POST	/api/opc/sessions	OPC 会话管理	无
GET/POST	/api/opc/sessions/:id/messages	OPC 消息管理	无
GET/POST	/api/llm/providers	LLM 模型配置	无
GET/POST	/api/prefs	用户偏好	无
GET/POST	/api/notifications	通知管理	无
GET	/health	健康检查	无

12. Electron 打包流程

12.1 打包配置

```
// package.json 中的 build 配置
{
  "build": {
    "appId": "com.stzh.video-workspace",
    "productName": "腾昇智和 Video Workspace",
    "directories": { "output": "dist" },
    "files": ["electron/**/*", "server/**/*", "out/**/*", "!node_modules"],
    "asar": true,
  }
}
```

```

    "asarUnpack": ["server/**/*", "out/**/*"],
    "win": {
      "target": [{ "target": "nsis", "arch": ["x64"] }]
    },
    "nsis": {
      "oneClick": false,
      "allowToChangeInstallationDirectory": true
    }
  }
}

```

12.2 构建命令

```

# 完整构建
npm run electron:build
# 等价于: npm run build && electron-builder --win

# 仅构建前端
npm run build
# 等价于: next build (静态导出到 out/)

# 开发模式
npm run electron:dev
# 等价于: electron .

```

12.3 Electron 主进程

```

// electron/main.js 核心流程
app.whenReady().then(async () => {
  // 1. 找空闲端口 (8080-8179)
  const port = await findFreePort(8080);

  // 2. 加载 .env.local
  loadDotenv(path.join(SERVER_DIR, ".env.local"));

  // 3. 设置数据目录
  process.env.STZH_DATA_DIR = app.getPath("userData");

  // 4. 启动 Express 后端
  const expressApp = require("./server-express.js");
  expressApp.listen(port);

  // 5. 打开窗口指向同一端口

```

```
createWindow(`http://localhost:${port}`);  
});
```

12.4 打包产物

```
dist/  
├── 腾昇智和 Video Workspace Setup 0.1.0.exe (~101 MB)  
├── win-unpacked/  
│   ├── 腾昇智和 Video Workspace.exe  
│   └── resources/  
│       ├── app.asar (打包的应用)  
│       └── app.asar.unpacked/  
│           ├── server/ (原生模块 + 后端代码)  
│           └── out/ (前端静态文件)
```

13. ECS 云服务器 + 外网部署

项目的默认分发方式是 Electron 桌面客户端（每人装一份），但为了满足“客户不想装软件、只想打开链接看看”的场景，额外实现了云服务器部署方案。这一章记录了从零开始在阿里云 ECS Windows 服务器上部署的完整过程，包括所有踩过的坑——从 native 模块编译到防火墙配置，每一步都有实际验证过的命令。

13.1 服务器配置

项目	值
平台	阿里云 ECS
操作系统	Windows Server
公网 IP	121.199.20.161
端口	8080
部署路径	C:\stzh\

13.2 前置依赖安装

1. **Node.js v25.x** — 必须和本地版本一致（避免 native 模块 ABI 不匹配）
2. **Python 3.12** — node-gyp 编译需要
3. **Visual Studio Build Tools** — C++ 编译器，选择 "Desktop development with C++"

13.3 部署命令（完整流程）

```
:: 1. 解压部署包
cd C:\stzh
powershell Expand-Archive -Path stzh-full.zip -DestinationPath . -Force

:: 2. 编译 native 模块 (better-sqlite3)
cd C:\stzh\node_modules\better-sqlite3
npm run node-gyp rebuild --release

:: 3. 安装 PM2 并启动
cd C:\stzh
npm install -g pm2
pm2 start start.js --name stzh
pm2 save

:: 4. 配置 Windows 防火墙
netsh advfirewall firewall add rule name="STZH 8080" dir=in action=allow
protocol=TCP localport=8080
```

13.4 阿里云安全组配置

入方向 → 手动添加：

- 协议类型：自定义 TCP
- 端口范围：8080/8080
- 授权对象：0.0.0.0/0

13.5 API 地址动态化

为支持外网访问，前端 API 地址从硬编码改为动态获取：

```
// 修改前
const API_BASE = process.env.NEXT_PUBLIC_AGENT_BACKEND_URL ||
"http://localhost:8080";

// 修改后
const API_BASE = process.env.NEXT_PUBLIC_AGENT_BACKEND_URL ||
window.location.origin;
```

修改涉及 5 个文件：auth.ts、sync.ts、server-sync.ts、opc-agent-api.ts、ChatFlow.tsx

13.6 静态文件路径修复

`server-express.js` 中的静态文件路径需要兼容两种部署模式：

```
// 修改前 (只兼容 Electron 嵌套目录)
const OUT_DIR = path.join(__dirname, "..", "out");

// 修改后 (自动检测)
const OUT_DIR = process.env.STZH_OUT_DIR
  || (fs.existsSync(path.join(__dirname, "out"))
    ? path.join(__dirname, "out")          // 云服务器：扁平结构
    : path.join(__dirname, "..", "out")); // Electron：嵌套结构
```

13.7 常见部署问题

问题	原因	解决方案
ERR_DLOPEN_FAILED	better-sqlite3 ABI 不匹配	确保本地和服务端 Node 版本一致，在服务器上 <code>npx node-gyp rebuild --release</code>
注册/登录 Failed to fetch	前端 API 地址写死 localhost	本地 <code>npm run build</code> 重新构建，上传新 <code>out/</code>
页面返回 Not found JSON	静态文件路径错误	检查 <code>OUT_DIR</code> 路径，确认 <code>out/</code> 目录存在
浏览器无法访问	防火墙/安全组未放行	配置阿里云安全组 + Windows 防火墙放行 8080 端口
PM2 进程消失	Windows 用户注销	只断开远程桌面，不要注销

13.8 日常维护命令

```
pm2 list          :: 查看进程状态
pm2 logs stzh     :: 实时日志
pm2 logs stzh --err :: 仅错误日志
pm2 restart stzh  :: 重启服务
pm2 stop stzh     :: 停止服务
pm2 flush stzh    :: 清空日志
pm2 kill          :: 杀掉 PM2 守护进程
```

13.9 更新部署流程

```
# 本地：重新构建前端
npm run build

# 上传到服务器
scp -r D:\fms688_stzh-Agent\out Administrator@121.199.20.161:C:\stzh\

# 服务器：重启
pm2 restart stzh
```

14. 开发过程中遇到的问题与解决方案

这一章记录了从立项到部署全过程中遇到的所有关键技术难题。每个问题都包含现象描述、根因分析、踩坑过程和最终解决方案。这些经验是用时间换来的，希望对后续维护者有参考价值。

14.1 Coze 知识库命中率低（RAG 阶段）

现象：用户输入“镜头慢慢推进”，知识库检索结果完全不相关，返回的是随机的图像参数。

根因：最初按照纯向量检索的思路设计知识库——把所有字段拼成一段文本，期望语义相似度匹配。但 Coze 表格知识库的底层机制是 **NL2SQL + 多路召回融合**，它会把用户查询转化为 SQL 条件去匹配列名和列值，而不是做语义模糊匹配。

踩坑过程：

- 第一版：把 Excel 所有列拼成一个“描述”列 → 命中率 < 20%
- 第二版：添加语义别名列 → 命中率 ~40%，但口语化查询仍然失败
- 第三版：开发 KB_Bridge 桥接层 → 命中率 > 85%

最终方案：

1. 重构知识库表结构：每行一个知识单元，每列一个语义维度，列名使用中英文双语
2. 开发 KB_Bridge 桥接层：三个 DOCX 文档，将口语化查询映射到 [MAP] 标准化术语
3. 两阶段检索：先查桥接层获取 slot_key，再用 slot_key 查表格知识库
4. 编写 17 个 Python 测试脚本（测试KB命中率.py、优化docx分片.py 等）持续迭代

14.2 连贯模式串行超时（ workflow 阶段）

现象：连贯模式生成 8 个镜头时，第 6 个镜头开始超时（Coze workflow 单节点有执行时间限制）。

根因：传统实现方式是串行生成——镜头 1 的视频生成完后才开始镜头 2。N 个镜头总耗时 = N × 单镜头耗时，N>5 时必然超时。而纯并行方案会导致后续镜头依赖前序镜头的结束帧，形成死锁。

踩坑过程：

- 串行方案：8 镜头耗时约 40 分钟，第 6 个超时
- 纯并行方案：后续镜头拿不到前序镜头的输出，死锁
- 分批串行：每 3 个一组并行，仍然超时

最终方案——“拉链拼接”架构：

1. **数据立交桥**：将图像描述和运镜指令分离为两个独立的数据流
2. **核心“变速箱”代码节点**：用滑动窗口将 N 张图配对为 N-1 个视频任务。镜头 i 的起始帧 = 图 i, 结束帧 = 图 i+1
3. **视频批量全并行渲染**：N-1 个视频任务零交叉依赖，全部并行执行

效果：8 镜头从 40 分钟降到约 8 分钟（单镜头耗时）。

14.3 Three.js 粒子性能（前端阶段）

现象：在集成显卡的笔记本上，粒子背景帧率低于 20fps，明显卡顿。

根因：初始实现使用了 `three/examples/jsm/postprocessing` 的 bloom 效果，每一帧需要额外的全屏后处理 pass，对 GPU 压力极大。

踩坑过程：

- 第一版：300 粒子 + UnrealBloomPass → 集成显卡 15fps
- 第二版：减少到 150 粒子 + 降低 bloom 分辨率 → 25fps，但视觉效果大打折扣
- 第三版：放弃 bloom，改用 AdditiveBlending → 60fps，视觉效果反而更好

最终方案：

1. 放弃 `postprocessing bloom`，改用 `THREE.AdditiveBlending` + 程序生成的软光晕 `CanvasTexture`
2. 三层粒子分离：150 冷色远景 + 100 中景 + 50 暖色近景，每层独立更新
3. 主题色跟随：从 CSS 变量读取 `--glow-warm / --glow-cool`，每 0.6s 刷新缓存
4. 无障碍降级：`prefers-reduced-motion` 激活时渲染静态渐变背景，关闭所有动画

14.4 CSS 自定义属性无法动画（UI 阶段）

现象：`edge-glow` 旋转锥形渐变需要 `--glow-angle` 从 `0deg` 变到 `360deg`，但 `transition: --glow-angle 4s` 无效。

根因：CSS 自定义属性（`--xxx`）默认是字符串类型，浏览器不知道如何对其做插值动画。

解决方案：使用 CSS Houdini 的 `@property` 注册属性类型：

```
@property --glow-angle {  
  syntax: "<angle>";      // 告诉浏览器这是角度类型  
  initial-value: 0deg;
```

```
inherits: false;
}
```

注册后, `--glow-angle` 就可以参与 `@keyframes` 动画了。这是整个 `edge-glow` 系统的基础。

兼容性注意: CSS Houdini 在 Firefox 和 Safari 中支持有限。降级方案是使用伪元素旋转代替。

14.5 Electron 打包后数据库无法写入 (打包阶段)

现象: Electron 打包的应用启动时报错 `unable to open database file`。

根因: `db.js` 中 `DB_PATH = path.join(__dirname, "stzh.db")`。在 Electron 打包后, `__dirname` 指向 `app.asar` 包内——这是一个只读的归档文件, SQLite 无法在里面创建或写入文件。

解决方案:

1. `electron/main.js` 中通过 `app.getPath("userData")` 获取可写目录 (通常在 `C:\Users\<用户>\AppData\Roaming\stzh-video-workspace\`)
2. 设置环境变量 `process.env.STZH_DATA_DIR = dataDir`
3. `db.js` 优先使用 `STZH_DATA_DIR`, 兜底检测 `__dirname` 是否在 `.asar` 内

14.6 better-sqlite3 Node ABI 版本不匹配 (打包 + 部署阶段)

现象: Electron 打包后启动报错 `NODE_MODULE_VERSION 133, This version requires NODE_MODULE_VERSION 141`。云服务器部署时同样的错误。

根因: `better-sqlite3` 是 C++ 原生模块, 编译时绑定特定的 Node.js ABI 版本。Electron 使用自己的 Node 版本 (v24, NMV 133), 而系统安装的 Node.js 是 v25 (NMV 141)。两者编译出的 `.node` 二进制文件不兼容。

踩坑过程:

- 在本地用 `npm rebuild better-sqlite3` → 编译为系统 Node 25 版本
- Electron 用 Node 24 加载 → ABI 不匹配
- 用 `@electron/rebuild` 重新编译 → Electron 正常, 但上传到服务器 (Node 25) 又不匹配

最终方案:

1. 本地开发/打包 Electron: 用 `npx @electron/rebuild -v 35.7.5` 编译
2. 云服务器部署: 确保服务器 Node 版本和本地一致 (v25), 然后在服务器上 `npx node-gyp rebuild --release`
3. 云服务器需要安装 Python + Visual Studio Build Tools 才能编译

14.7 server-express.js 缺少路由（打包阶段）

现象：Electron 打包的应用登录/注册报错 `Unexpected token '<', "<!DOCTYPE"... is not valid JSON`。

根因：项目有两套 Express 服务器——`server.js`（独立部署用，挂载了所有路由模块）和 `server-express.js`（Electron 用，最初是精简版）。`server-express.js` 只有 `/api/agent`、`/api/opc/*`、`/api/llm/*` 等内联路由，完全缺少 `auth`、`conversations`、`templates`、`generations` 路由和 JWT 中间件。前端请求 `/api/auth/login` 时，Express 没有这个路由，落到 `catch-all` 中间件返回了 `index.html`，前端 `response.json()` 解析 HTML 就报错了。

解决方案：

```
// 在 server-express.js 中添加 JWT 中间件和路由模块
const jwt = require("jsonwebtoken");
const { JWT_SECRET } = require("./routes/auth.js");

app.use((req, _res, next) => {
  req.user = null;
  const authHeader = req.headers.authorization;
  if (authHeader?.startsWith("Bearer ")) {
    try { req.user = jwt.verify(authHeader.slice(7), JWT_SECRET); } catch {}
  }
  next();
});

app.use(require("./routes/auth.js"));
app.use(require("./routes/conversations.js"));
app.use(require("./routes/templates.js"));
app.use(require("./routes/generations.js"));
app.use(require("./routes/settings.js"));
```

14.8 settings 路由冲突（部署阶段）

现象：挂载 `routes/settings.js` 后，应用级设置（主题等）读取报错。

根因：`server-express.js` 有内联的 `GET/POST /api/settings` 路由（操作 `app_settings` 表，key-value 存储），而 `routes/settings.js` 也有 `GET/PUT /api/settings` 路由（操作 `user_settings` 表，需要 JWT 鉴权）。Express 按注册顺序匹配，先注册的路由会拦截所有同路径请求。两者操作不同的表、使用不同的 HTTP 方法、返回不同的数据格式，但路径完全相同。

解决方案：将内联的应用级设置路由重命名为 `/api/app-settings`，更新前端 `server-sync.ts` 中三处调用路径。这样用户设置走 `/api/settings`（需要登录），应用设置走 `/api/app-settings`（无需登录），互不干扰。

14.9 前端 API 地址硬编码（部署阶段）

现象：云服务器部署后，用户通过 `http://121.199.20.161:8080` 打开页面，页面能加载但注册/登录报错 `Failed to fetch`。

根因：5 个前端文件（`auth.ts`、`sync.ts`、`server-sync.ts`、`opc-agent-api.ts`、`ChatFlow.tsx`）中 API 地址写死为 `process.env.NEXT_PUBLIC_AGENT_BACKEND_URL || "http://localhost:8080"`。由于 `output: "export"` 静态导出，环境变量在构建时被烘焙进 JS 文件。用户浏览器访问外网地址时，前端 JS 仍然尝试请求 `http://localhost:8080` ——请求的是用户自己电脑上的 localhost，而不是服务器。

解决方案：将 fallback 从硬编码地址改为 `window.location.origin`：

```
const API_BASE = process.env.NEXT_PUBLIC_AGENT_BACKEND_URL ||
window.location.origin;
```

这样无论从哪个地址访问，API 请求都会发到当前页面的同一 origin。本地 `localhost:8080`、外网 `121.199.20.161:8080`、ngrok URL 都能自动适配。

14.10 静态文件路径不兼容（部署阶段）

现象：云服务器上 `http://121.199.20.161:8080` 返回 `{"error":{"message":"Not found"}}`，而不是前端页面。

根因：`server-express.js` 中 `const OUT_DIR = path.join(__dirname, "..", "out")`。在 Electron 打包模式下，目录结构是嵌套的（`app.asar/server/` → `../out`），路径正确。但在云服务器上，所有文件平铺在 `C:\stzh\` 下，`__dirname` 就是 `C:\stzh`，`..` 变成了 `C:\out`——不存在的路径。Express 找不到静态文件，请求落到 catch-all 返回 JSON 404。

解决方案：自动检测目录结构：

```
const OUT_DIR = process.env.STZH_OUT_DIR
|| (fs.existsSync(path.join(__dirname, "out"))
  ? path.join(__dirname, "out") // 扁平结构（云服务器）
  : path.join(__dirname, "..", "out")); // 嵌套结构（Electron）
```

14.11 SSE 流式解析缓冲区碎片（后端阶段）

现象：SSE 流偶尔报 JSON parse error，频率约 1/50 次。

根因：TCP 流是字节流，不保证消息边界。一个 SSE 事件（`event: xxx\ndata: {...}\n\n`）可能被拆成多个 TCP 包。直接对每个包做 `JSON.parse` 会遇到不完整的 JSON 片段。

解决方案：实现缓冲区累积 + 双换行分割：

```

let buffer = "";
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  const events = buffer.split("\n\n"); // SSE 事件以双换行分隔
  buffer = events.pop() || ""; // 最后一个可能是不完整的，留到下次
  for (const event of events) {
    // 解析完整的事件
  }
}

```

14.12 Coze 卡片响应格式变更（后端阶段）

现象：知识库检索结果突然无法解析，返回空结果。

根因：Coze 平台更新了卡片响应格式，从直接返回 JSON 变为嵌套的 `card_type` + `data` + `variables` 结构，且 `data` 字段可能是 JSON 字符串而非对象。

解决方案：编写 `parseCozeCard` 函数，兼容多种格式：

1. 先检测 `card_type` 字段判断是否为卡片格式
2. 如果 `data` 是字符串则二次 `JSON.parse`
3. 遍历 `variables` 中的 `defaultValue` 数组
4. 对每个 `item.【】` 尝试 JSON 解析，失败则作为纯文本处理

14.13 localStorage 容量限制（前端阶段）

现象：长期使用后，对话历史保存失败，控制台报 `QuotaExceededError`。

根因：localStorage 限制约 5MB。每个对话包含完整的消息历史（含 Prompt、响应文本、图片 URL 等），10 个对话就可能超过限制。

解决方案：

1. 消息裁剪：每个会话限制最大消息数（可配置 100/300/500/1000）
2. 服务端优先：登录后对话历史存储到 SQLite，localStorage 仅作为离线缓存
3. 导出功能：支持 Markdown/JSON/txt 格式导出，用户可以清理旧对话

14.14 Electron 端口冲突（打包阶段）

现象：第二次启动 Electron 应用时报错 `EADDRINUSE`。

根因：第一次启动时 Express 监听了 8080 端口。应用关闭时 Express 服务器没有完全释放端口（Windows 的 TIME_WAIT 状态），第二次启动时端口仍被占用。

解决方案：动态端口发现，从 8080 开始扫描到 8179：

```
async function findFreePort(start) {
  for (let p = start; p < start + 100; p++) {
    if (await isPortFree(p)) return p;
  }
  throw new Error("找不到可用端口");
}
```

前端窗口加载 `http://localhost:${动态端口}`，API 请求通过 `window.location.origin` 自动适配。

14.15 SplashScreen Canvas 性能（前端阶段）

现象：恒星坍缩动画在低端设备上掉帧，特别是粒子爆发阶段（160 粒子 + 200 像素碎片同时渲染）。

根因：Canvas 2D 每帧需要重绘所有粒子，500 颗星 + 360 粒子 + 6 个星云团 + 扫描线叠加，绘制调用过多。

解决方案：

1. 像素网格对齐：所有坐标 snap 到 6px 网格，减少亚像素渲染开销
2. 离屏 Canvas 缓存：将不常变化的元素（星云、扫描线）预渲染到离屏 Canvas
3. 粒子池化：预分配粒子对象数组，避免 GC 压力
4. 字符图集缓存：「腾昇智和」四字的像素化结果缓存到 Map，不重复计算

14.16 聊天输入框焦点丢失（前端阶段）

现象：发送消息后，输入框偶尔失焦，需要点击才能继续输入。

根因：发送消息后触发了多个状态更新（消息列表滚动、星环脉冲动画、思维阶段切换），某些更新导致 React 重渲染，输入框的 `focus` 状态被清除。

解决方案：

1. 发送后显式调用 `inputRef.current?.focus()`
2. 使用 `useCallback` 稳定事件处理函数引用
3. 将动画触发与输入框状态解耦（`burstKey` 单独管理）

14.17 multer 文件上传与 JSON 混合（后端阶段）

现象：带文件上传的请求偶尔解析失败，`req.body` 为空。

根因：前端发送文件时使用 `multipart/form-data`，但 Express 的 `express.json()` 中间件只解析 `application/json`。两种 Content-Type 需要不同的中间件。

解决方案：

```
// server.js 中同时配置两种中间件
app.use(express.json({ limit: "10mb" }));
const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 50
* 1024 * 1024 } });

// agent 路由使用 multer 处理文件
app.post("/api/agent", upload.array("files", 10), async (req, res) => {
  // req.body 由 multer 解析 (兼容 JSON 和 form-data)
  // req.files 包含上传的文件
});
```

14.18 主题切换时布局抖动（UI 阶段）

现象：切换主题时，页面元素有短暂的闪烁/跳动。

根因：主题切换通过修改 `document.documentElement.dataset.theme` 触发 CSS 变量重新计算。在变量更新的瞬间，部分元素的尺寸/颜色处于中间状态。

解决方案：

1. CSS 变量定义时添加 `transition: background-color 0.3s, color 0.3s` 平滑过渡
2. 主题切换前添加 `document.documentElement.style.setProperty('color-scheme', 'dark')` 提示浏览器
3. 关键布局元素使用固定的 `min-height` 避免内容高度变化导致的跳动

14.19 React Context 重渲染风暴（前端阶段）

现象：修改一个偏好设置时，整个页面所有组件都重新渲染。

根因：所有偏好设置放在单个 React Context 中，任何值变化都会触发所有消费者重渲染。

解决方案：

1. 拆分 Context：将偏好设置按功能拆分为多个独立 Context（主题、通知、聊天设置等）
2. 使用 `useMemo` 缓存 Context value

3. 组件使用 `React.memo` 包裹，仅在相关 props 变化时重渲染

14.20 windows-build-tools 安装失败（部署阶段）

现象：在云服务器上执行 `npm install -g windows-build-tools` 报错，尝试从淘宝镜像下载 Python 2.7 失败（404）。

根因：`windows-build-tools` 包已废弃（deprecated），其依赖的 Python 2.7 下载链接已失效。

解决方案：手动分别安装：

1. Python 3.12：从 python.org 下载 `.exe` 安装包（不是 `.tgz` 源码包），安装时勾选 "Add to PATH"
2. Visual Studio Build Tools：从微软官网下载 Build Tools for Visual Studio 2022，选择 "Desktop development with C++" 工作负载

14.21 部署包目录结构被打平（部署阶段）

现象：上传 zip 到服务器解压后，`routes/` 和 `start.js` 在同一层级，没有 `server/` 父目录。

根因：PowerShell 的 `Compress-Archive` 在压缩时会打平指定的路径结构。指定 `server\start.js` 时，压缩包内只保留文件名 `start.js`，不保留 `server\` 前缀。

解决方案：

1. 接受扁平结构——代码中的相对路径（`require("../routes/auth")`）在同一目录下仍然有效
2. 调整 `OUT_DIR` 检测逻辑兼容两种结构
3. 部署文档中明确说明解压后的目录结构

14.22 PM2 进程在 Windows 上不稳定（部署阶段）

现象：Windows 远程桌面注销后 PM2 进程消失，或 `pm2 restart` 报 "Process not found"。

根因：PM2 在 Windows 上的守护进程依赖用户会话。注销用户会话时，PM2 daemon 随之终止。此外，Windows 更新或休眠也可能导致 PM2 daemon 异常退出。

解决方案：

1. 远程桌面只"断开连接"，不"注销"
2. 使用 `pm2 kill` 后重新 `pm2 start` 恢复
3. 配置 Windows 计划任务，在系统启动时自动运行 PM2（替代 Linux 的 `pm2 startup`）

总结：22 个问题中，有 10 个发生在部署阶段（14.5-14.10、14.20-14.22），说明**部署环节是全链路中最脆弱的部分**。根本原因是 Windows 生态下原生模块编译的复杂性，以及 Electron 和云服务器两种运行环境的差异。后续如果需要大规模分发，建议将后端部署到 Docker 容器中，彻底消除环境差异。

附录

A. 项目文件结构

```
fms688_stzh-Agent/
├── app/
│   ├── api/agent/route.ts           - Coze API 代理
│   ├── api/agent/stream/route.ts   - SSE 流式代理
│   ├── api/search/route.ts         - 搜索代理
│   ├── components/
│   │   ├── SplashScreen.tsx        - 恒星坍缩首页
│   │   ├── HomeClient.tsx          - 根组件（路由+状态+背景）
│   │   ├── ChatFlow.tsx            - 对话主组件
│   │   ├── ChatInput.tsx           - 悬浮胶囊输入
│   │   ├── ResultCard.tsx          - 视频/图片结果卡片
│   │   ├── OPCPanel.tsx            - OPC 创作助手
│   │   ├── StarfieldBackground.tsx - Three.js 粒子
│   │   ├── OrbitRings.tsx          - CSS 星环
│   │   ├── CursorTrail.tsx         - 鼠标拖尾
│   │   ├── SettingsDrawer.tsx      - 设置面板
│   │   ├── LeftSidebar.tsx         - 左侧栏
│   │   ├── StatsDashboard.tsx      - 数据统计
│   │   └── LayerStack.tsx          - Z 轴背景层
│   ├── lib/
│   │   ├── auth.ts                 - 鉴权库
│   │   ├── sync.ts                 - 同步层
│   │   ├── server-sync.ts          - 服务端同步
│   │   ├── opc-agent-api.ts        - OPC API
│   │   └── rag-client.ts           - RAG 客户端
│   ├── globals.css                 - 设计系统 (1336 行)
│   ├── layout.tsx                  - 根布局
│   └── page.tsx                     - 入口页
├── server/
│   ├── server.js                   - 独立 Express 服务器
│   ├── server-express.js           - Electron Express 服务器
│   ├── db.js                       - SQLite 数据库层
│   ├── start.js                    - 云服务器启动入口
│   └── routes/
│       └── auth.js                 - 注册/登录/验证
```



B. 外部依赖

依赖	版本	用途
Next.js	16.2.4	前端框架
React	19.2.4	UI 库
Tailwind CSS	4	样式系统
Three.js	0.184	3D 粒子
GSAP	3.15	动画编排
Express	5.1.0	后端框架
better-sqlite3	12.10.0	数据库
bcryptjs	-	密码哈希
jsonwebtoken	-	JWT 鉴权
Electron	35.7.5	桌面客户端
electron-builder	26.8.1	打包工具

C. 参考文档

- Coze 知识库调用原理： README/Coze知识库调用README.md
- 技术文档： tech_doc.md
- 系统提示词： README/最终版人设与逻辑.md
- 混剪模式提示词： README/混剪模式系统提示词.md

- 连贯模式提示词: `README/连贯模式系统提示词.md`
- 部署指南: `deploy/DEPLOY.md`